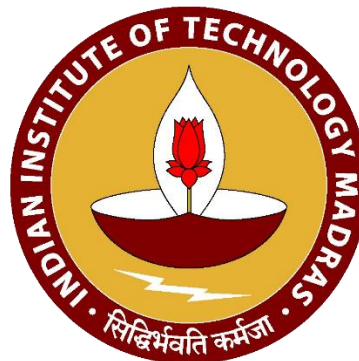


# **Python for Data Science**

## **Web M. Tech Course DA5301W**

### **Module 12: AI Tools for Assisted Programming**

**Dr. P.S Jayadev**



# Contents

- Gen AI and its Modalities
- Text and Code Generation
- AI Agents
- AI Assisted Programming

# Gen AI and its Modalities

- **Generative AI:** Intelligence that is capable to generate new content (text, code, images, audio, video) given prompts or context
- **How it works:** Learns patterns in large datasets and predicts plausible continuations i.e., next word, next pixel, next video frame or next audio frame
- **Modes of Gen AI:**
  - Text: Summarization, Q&A, Translation, Speech to Text, etc.
  - Code: Completion, Refactoring, Documentation, debugging, etc.
  - Images: Concept art, Branding, Presentations, edits/variations.
  - Audio: Voice generation, Cloning, Sound design, Composing Music, etc.
  - Video: Clip generation, Video editing, Graphics, Captioning, etc.

# Text and Code Generation

# Large Language Models (LLMs)

- Language Model predicts the most probable next word in a sequence given the previous words
- LLMs are advanced models trained to learn patterns from massive datasets of text or code
- LLMs do not know “facts” like humans but based on a prompt, they can generate:
  - A sentence or paragraph
  - A function or entire script
  - Explanations, summaries, etc.
- **Note:** Their responses are probabilistic, not guaranteed to be correct

# How do LLMs understand Language?

- LLMs cannot directly process raw text or code
- Human-readable input must be converted into numerical form
- This happens in two key steps:
  - Tokenisation
  - Vectorisation (Embedding)

# Tokenisation

- Process of splitting text or code into smaller units called tokens
- Tokens are the basic units processed by a language model
- Tokens can represent:
  - Whole words: "data"
  - Subwords: "token" + "ization"
  - Characters or symbols: {, (, ==
  - Numbers or fragments of numbers: "202", "5"
  - Whitespace or line breaks (important for code)
- Each LLM is trained with a specific tokenizer and a fixed vocabulary
- The model can only process and generate tokens from this vocabulary

# Classical Tokenisation Techniques

- **Whitespace Tokenisation:** Splits text using spaces
  - Eg: "AI helps developers" → ["AI", "helps", "developers"]
- **Punctuation-Based Tokenisation:** Separates words and symbols using punctuation rules
- **Regular Expression Tokenisation:** Uses handcrafted rules for splitting text
- **Sentence Tokenisation:** Splits text into sentences using language-specific rules
- **Characteristics:**
  - Simple Rule-based and deterministic
  - Language-dependent
  - Limited ability to handle new or rare words



# Modern Tokenisation Techniques

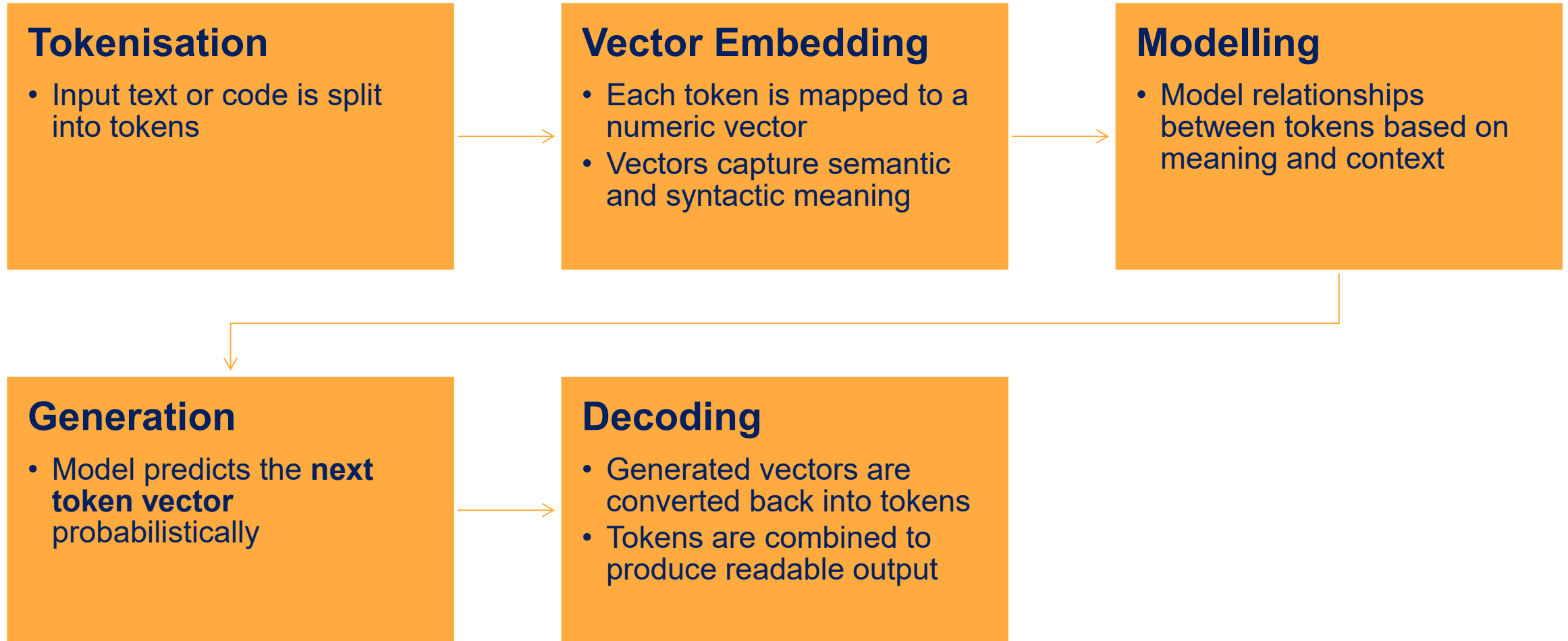
## ➤ Subword Tokenisation

- Breaks text into meaningful sub-parts of words instead of full words
- Learns these sub-parts from data, not from fixed rules
- Avoids the out-of-vocabulary (OOV) problem
- Handles rare words, technical terms, new variable names in code, etc.
- Eg: "tokenization" : ["tok", "en", "iz", "ation"]

## ➤ Byte level Tokenisation

- Splits input into raw bytes (0–255) instead of characters or words
- Treats text as a sequence of bytes
- Can represent any text, any language, emojis, etc.
- Eg: "help" : [104, 101, 108, 112]
- Guarantees that any input can be represented but token sequences are longer

# LLM Workflow: Text/Code Generation



# Context Window in LLMs

- The maximum number of tokens a language model can process at once
  - Includes:
    - System instructions
    - User prompt
    - Conversation history
    - Model-generated output
  - Examples:
    - GPT-5 context window: 400,000 tokens
    - Gemini 3 pro: 1,000,000 tokens
- Model may not perfectly recall or reason on information outside the context window
- Large codebases or notebooks often exceed context limits — important constraints may be ignored unless restated

# Halluniation in LLMs

- Occurs when an LLM generates plausible-sounding, fluent but **incorrect, fabricated, or misleading** information
- **Factual Hallucination:**
  - The model invents or alters facts, names, or numbers
  - **Why it happens:** lack of factual grounding or outdated training data.
  - **How to reduce:** Ask for sources and reasoning or Ask it to search
- **Logical Hallucination:**
  - Model's reasoning chain or explanation is illogical or inconsistent
  - **Why it happens:** reasoning drift, token prediction errors, or missing context
  - **How to reduce:** Use step-by-step prompts and specify “think aloud and verify logic.”
- **Key Takeaway:** LLMs can sound confident even when they are wrong — verification is the user's responsibility

# LLM Powered AI Agents

# LLM Powered AI Agents

- An AI agent is a system that perceives its environment, decides on an action, and then acts to achieve a goal — often in a loop
- LLM Powered Agents: An agent that combines LLM capabilities with to achieve the goal
- ChatGPT, Gemini, Perplexity, Cursor, etc. are AI Agents powered by LLMs

# Components of LLM Powered AI Agents

|                 |                                                                          |
|-----------------|--------------------------------------------------------------------------|
| <b>LLM Core</b> | Generates responses, performs reasoning, follows instructions            |
| <b>Tool Use</b> | Integrates with external APIs, databases, calculators, web, code runners |
| <b>Memory</b>   | Stores context across turns, sessions, or tasks                          |
| <b>Planning</b> | Breaks complex tasks into steps (e.g., Chain-of-Thought, ToT)            |
| <b>Autonomy</b> | Acts across multiple steps with minimal or no human intervention         |

# AI Assisted Programming



# Why Use AI tools in Programming?

## Reduces Learning Curve

- Help beginners understand programming concepts faster by offering examples, explanations, and suggestions.

## Saves Time

- Speeds up tasks like writing boilerplate code, debugging, or searching for syntax

## Improves Code Quality

- Suggests best practices in coding and, helping users write cleaner, more efficient code.

## Enhances Problem Solving

- AI tools can help break down a problem, suggest an approach, or convert a natural language problem into code.

## Adapts to Skill Level

- AI can adjust its responses based on the skill level of the user when prompted

# Where AI Assists in Programming

## Code Writing

- Write entire functions or small scripts based on a natural language prompt

## Code Completion

- Predict and autofill the next part of your code

## Code Understanding

- Explain what a piece of code does in plain English

## Debugging

- Identify syntax or logic errors and suggest fixes

## Testing

- Generate test cases or write test functions for your existing

## Documentation

- Auto-generate or improve docstrings, comments, and README content

## Code Translation

- Convert logic or code from one programming language to another (e.g., C++ to Python)

# AI Tools for Programming Assistance

## Chat-based tools

ChatGPT,  
Deepseek,  
Gemini,  
Claude

## IDE-integrated tools

GitHub  
Copilot,  
Cursor,  
Continue  
Dev

## Testing tools

TestGPT,  
Codium AI

## Documentation tools

ExplainDev,  
Cithub  
Copilot

# Risks and Limitations of AI-Assisted Programming

## **AI optimizes for plausibility, not correctness**

- Code may look correct but contain logical, numerical, or API-level errors

## **False sense of productivity**

- Faster code generation can hide code duplication, poor design choices, accumulating technical debt

## **Hallucinations extend to code**

- Non-existent functions, deprecated libraries, incorrect assumptions

## **Weak at edge cases and system-level reasoning**

- Especially for complex workflows, integrations, or architecture decisions

## **Context limitations**

- Large codebases or long conversations may cause important constraints to be ignored

# Best Practices for Using AI in Programming

## **Keep humans in the loop**

- Always review AI-generated code before acceptance or deployment

## **Break problems into small, well-defined tasks**

- AI performs best on narrow, clearly scoped requests

## **Be explicit in prompts**

- Specify constraints, assumptions, inputs, and expected outputs

## **Test and validate systematically**

- Run, test, and reason about the code — especially for critical logic

## **Use AI as a learning and productivity aid**

- Treat outputs as drafts or suggestions, not final answers

## **Rely on programming fundamentals**

- Understanding code is essential to detect errors, adapt solutions, and debug issues

# Final Takeaways

- AI can accelerate development, but it can also silently degrade code quality if used blindly
- While AI can be used to write code faster, developers are still responsible for correctness, design, and maintainability